

MACHINE LEARNING WITH AI AND PYTHON

DAY 3: BUILDING WITH LLMS, EMBEDDINGS, RAG, AND AGENTS



LEARNING OBJECTIVES

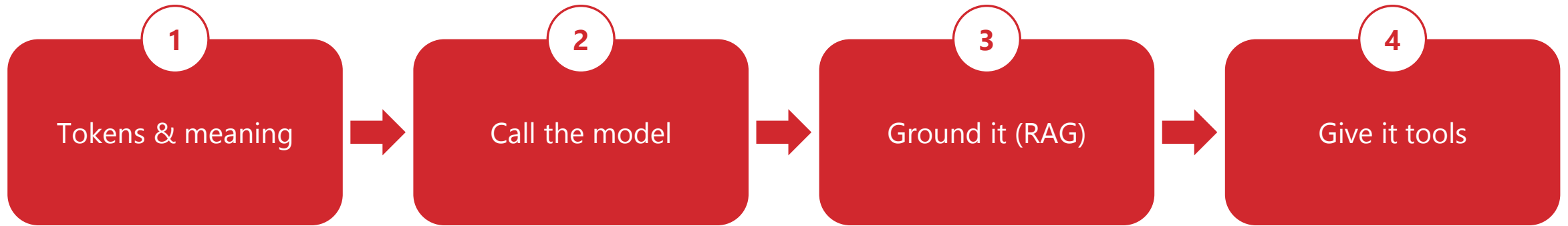
Days 1 and 2 built models from data. Day 3 is about building with large language models: calling them, grounding them in your own documents, and letting them take real actions.

- Explain how an LLM turns text into tokens and meaning into vectors
- Measure semantic similarity and build search by meaning
- Call foundation models on AWS Bedrock with the Converse API
- Build a RAG pipeline that answers from your own documents
- Give a model tools so it can read and write live data
- Route each question to the right model and data source



YOUR DAY 3 ROADMAP

Four skills, built in order. Each one depends on the one before it, and together they add up to a working AI assistant.



By this afternoon you will have built a support assistant that reads manuals and checks live inventory.

WHY THESE FOUR SKILLS, IN THIS ORDER

A foundation model out of the box is fluent but ungrounded: it knows the internet up to a training date, nothing about your business, and it cannot look anything up. Everything today closes that gap.

The gap you are closing

- It reads tokens, not words, so cost and limits behave in non-obvious ways
- It has no access to your documents, so it guesses when it should not
- It cannot act: no database reads, no tickets, no live data

The through-line:

Tokens and embeddings are the foundation. Bedrock is the connection. RAG is the knowledge. Tools are the hands.

MODULE 1: HOW LLMS READ TEXT

TOKENS AND EMBEDDINGS



START WITH HOW A MODEL SEES TEXT

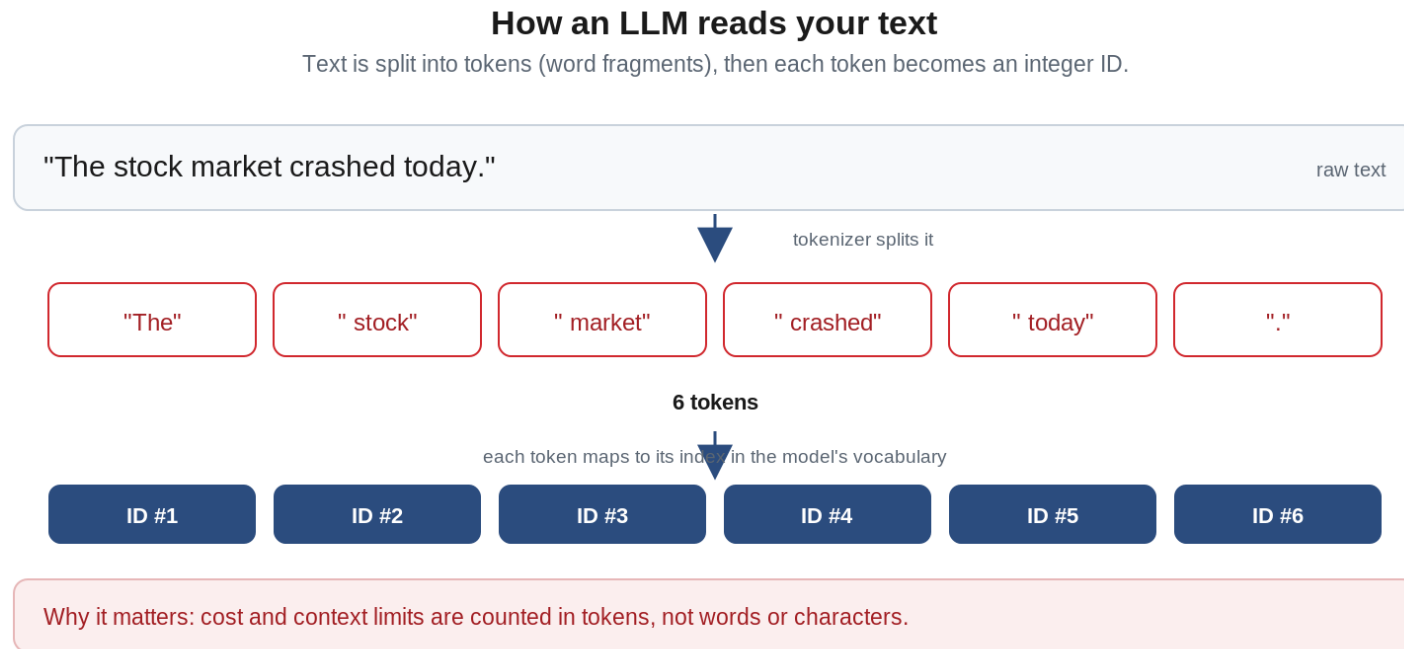
An LLM does not read letters or words, and it does not understand meaning through grammar. It reads tokens, and it captures meaning as vectors. These two ideas are the bedrock under every lab today.

Two ideas, everything else builds on them

- Tokenization: text split into fragments the model can process
- Embeddings: meaning turned into a list of numbers you can compare
- The RAG pipeline this afternoon is embeddings end to end

WHAT AN LLM ACTUALLY READS

The tokenizer is the first thing every prompt hits, before any intelligence happens. It splits your text into tokens and turns each into an integer ID.



Common words are usually one token. Rare or long words get split into several.

TOKENS DRIVE COST AND CONTEXT LIMITS

Two hard limits in every LLM app are measured in tokens, not words: what you pay, and how much the model can hold at once. Counting tokens before you send is how you stay ahead of both.

What tokens control

- Cost: APIs bill per token, for input and output separately
- Context window: the maximum tokens the model can consider at once
- Truncation: go over the window and content silently drops

Why this matters:

A modern model can hold hundreds of thousands of tokens, so cost only bites at scale. But a long retrieved context can still overflow the window.

COUNT TOKENS BEFORE YOU SEND

Counting tokens is a two-line habit that tells you the cost of a call and whether it will fit, before you spend anything.

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")    # OpenAI GPT-4 encoding

def count_tokens(text):
    return len(enc.encode(text))

prompt = "Explain gradient descent, step by step."
n = count_tokens(prompt)
cost = n / 1_000_000 * 0.80                # example rate: $0.80 per 1M tokens
print(n, "tokens", f"${cost:.6f}")
```

Use case: estimate a prompt's size and confirm it fits the window before sending.

SUBWORD TOKENIZATION: WHY WORDS SPLIT

A tokenizer does not keep a separate token for every possible word; that list would never end. It keeps common words and common fragments, then builds rare words out of those pieces. That is why no word is ever truly unknown to the model.

How it plays out

- Common words like 'the' or 'market' are a single token
- Rare or long words break into familiar fragments
- A brand new word still tokenizes, just into more pieces

The idea:

This scheme is called byte pair encoding. It is why 'cat' is one token while a long invented word costs many.

WHAT INFLATES YOUR TOKEN COUNT

Two prompts that look about the same length can cost very different amounts. Knowing what runs up the count helps you write leaner prompts and estimate cost honestly.

Sneaky cost drivers

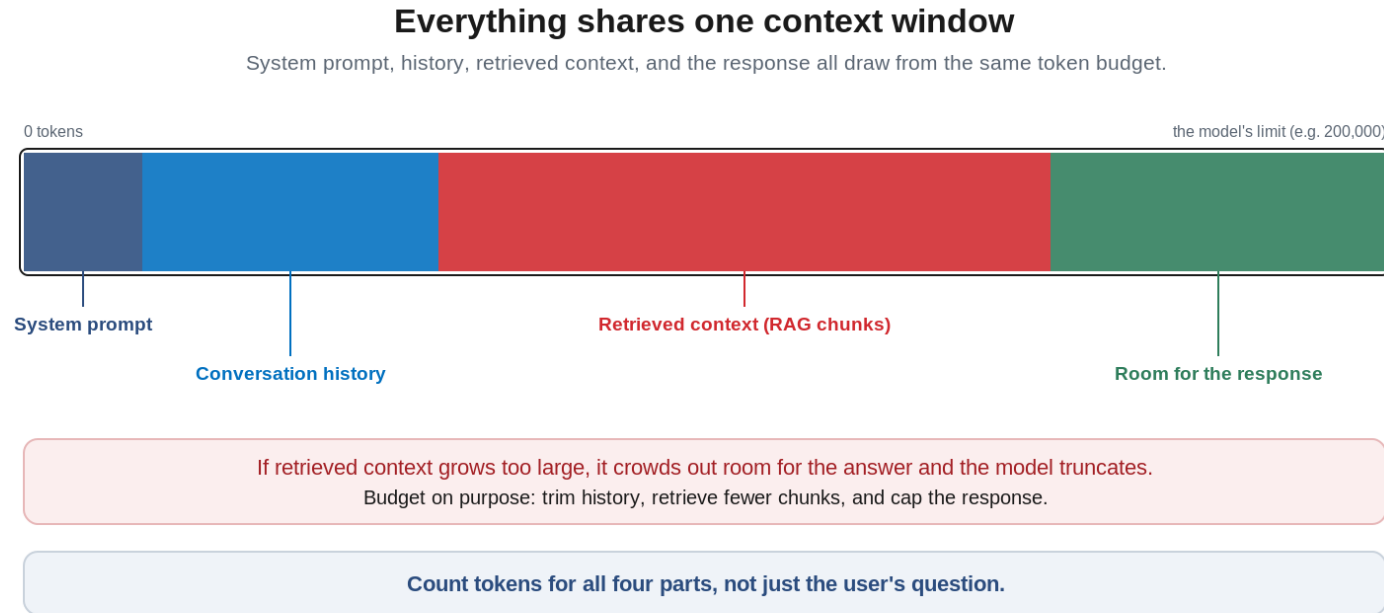
- Whitespace, line breaks, and markdown formatting all become tokens
- Numbers and long IDs often split into several tokens
- Code is token dense: punctuation and indentation add up fast
- Non-English text usually costs more tokens per word

Watch out:

In a RAG app the retrieved context is usually the biggest cost driver, not the user's question.

MANAGING THE CONTEXT WINDOW

The context window is a fixed budget shared by everything you send plus the response. Treat it like memory: if one part balloons, another gets squeezed out.



Trim history, retrieve fewer chunks, and cap the response to stay inside the budget.

POP QUIZ: TOKENS

You rewrite a prompt to be more polite and add two worked examples. Which goes up, cost or the context window, and which of those can you actually control?



2 MINUTES

POP QUIZ: TOKENS

You rewrite a prompt to be more polite and add two worked examples. Which goes up, cost or the context window, and which of those can you actually control?

Answer:

Adding text raises your token count, so input cost goes up and you use more of the window. The window size is fixed by the model.

- More tokens in means more you pay and less room left for a response
- You control prompt length and output length; you do not control the window
- Trim examples or retrieved context first when you need to fit



FROM TOKENS TO MEANING: EMBEDDINGS

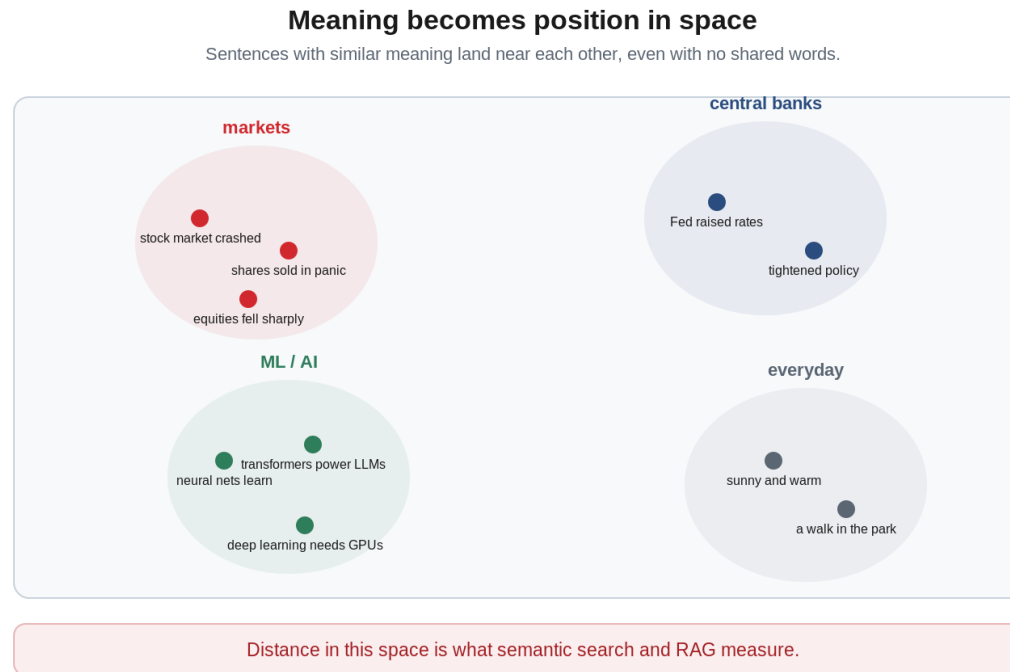
Token IDs are just indexes: ID 5 is not more like ID 6 than ID 500. To compare meaning, you need embeddings, which place text in a space where distance means similarity.

What an embedding is

- A dense vector of floats, often 384 or 1536 numbers
- Produced by a model trained so similar meanings land nearby
- The same idea in different words gives nearly the same vector

MEANING BECOMES POSITION

Sentences about the same topic cluster together, even when they share no words. The two everyday sentences sit far from the finance and ML groups.



Distance in this space is exactly what semantic search and RAG measure.

ENCODE TEXT AS VECTORS

One model call turns any list of sentences into vectors you can compare. The download happens once, then it runs locally.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2") # small, fast, solid

sentences = [
    "The Federal Reserve raised interest rates.",
    "Central bank increases borrowing costs.",
    "The cat sat on the mat.",
]
embeddings = model.encode(sentences)
print(embeddings.shape) # (3, 384): 384 numbers per sentence
```

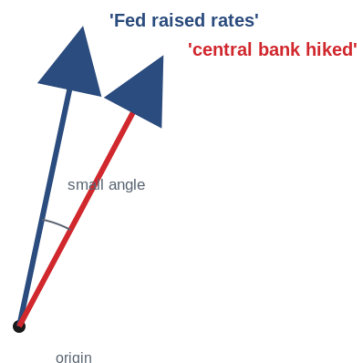
Use case: turn any text into a vector you can search or cluster by meaning.

COSINE SIMILARITY: HOW CLOSE IS CLOSE

To compare two embeddings you measure the angle between them. A score near 1 means the same meaning, near 0 means unrelated, below 0 means opposing.

Cosine similarity: how close is close?

Compare two embeddings by the angle between them, not their length.



similarity score

1.0

identical meaning

0.85

'stocks fell' vs 'equities declined'

0.30

loosely related

0.02

'interest rates' vs 'the cat sat'

< 0

opposite / unrelated

This one operation powers every semantic search and RAG retrieval.

This single operation sits at the heart of every semantic search and RAG retrieval.

COMPARE BY MEANING

Cosine similarity scores how close two meanings are. Watch it stay high for paraphrases with no shared words, and drop for unrelated text.

```
from sklearn.metrics.pairwise import cosine_similarity

a = model.encode(["Stock prices fell sharply."])
b = model.encode(["Equity markets declined today."])
c = model.encode(["The weather is warm today."])

print(cosine_similarity(a, b)[0][0])    # high: same meaning, no shared words
print(cosine_similarity(a, c)[0][0])    # low: unrelated
```

Use case: rank how relevant any candidate text is to a query.

SEMANTIC SEARCH: RETRIEVE BY MEANING

Put encoding and cosine similarity together and you get search that ignores keywords entirely: embed the query, compare it to every document, and return the closest matches.

The retrieval loop

- Embed the query with the same model as the documents
- Score it against every stored embedding with cosine similarity
- Sort and return the top-k highest scoring documents

Where this goes:

This is the retrieval half of RAG. In Module 3 you do the same thing at scale with a managed vector store.

SEARCH BY MEANING, NOT KEYWORDS

A dozen lines gives you a working semantic search. The query 'market selloff' finds the stock sentences even though those exact words never appear.

```
import numpy as np

def semantic_search(query, corpus, corpus_embeddings, top_k=3):
    q = model.encode([query])
    scores = cosine_similarity(q, corpus_embeddings)[0]
    ranked = np.argsort(scores)[::-1][:top_k]
    return [(scores[i], corpus[i]) for i in ranked]

# "market selloff" surfaces the stock sentences, no words in common
for score, doc in semantic_search("market selloff", corpus, embeddings):
    print(f"[{score:.3f}] {doc}")
```

Use case: this is the core retrieval step inside every RAG system.

CHOOSING AN EMBEDDING MODEL

The embedding model decides how well meaning is captured, so it matters as much as the language model in a search or RAG system. Bigger is not always better; you are trading quality for speed and storage.

What to weigh

- Dimensions: MiniLM gives 384, mpnet gives 768, larger models more
- More dimensions can mean better quality, but slower and heavier to store
- Domain matters: a model tuned on legal or medical text can beat a bigger general one

Non-negotiable:

Embed your documents and your queries with the same model. Mixing models makes the vectors incomparable.

COSINE, DISTANCE, AND KNOWING WHEN NOTHING FITS

Cosine similarity is the standard because it compares direction, not length, so a short sentence and a long one about the same thing still match. Just as important is recognizing when nothing matches.

Two practical points

- Cosine ignores magnitude; Euclidean distance is swayed by it, usually not what you want
- Every result comes with a score, so you can set a floor
- If the best score is low, the honest answer is 'I don't have that'

Why this matters:

An out-of-domain question should return low scores. Handle that case instead of forcing a weak match into the answer.

POP QUIZ: SEMANTIC SEARCH

A search for 'market selloff' returns documents that never contain the words 'market' or 'selloff'. How is that possible, and what does it let you do that keyword search cannot?



2 MINUTES

POP QUIZ: SEMANTIC SEARCH

A search for 'market selloff' returns documents that never contain the words 'market' or 'selloff'. How is that possible, and what does it let you do that keyword search cannot?

Answer:

The query and every document are turned into meaning vectors, and matches are ranked by how close those vectors are, not by shared words.

- 'market selloff' lands near 'shares sold in panic' in embedding space
- Users find answers without guessing the document's exact wording
- The trade-off: a topically close match can still be factually wrong



MODULE 2: CALLING LLMS ON BEDROCK

ONE API, MANY MODELS



WHAT AWS BEDROCK GIVES YOU

Bedrock is a managed service for calling foundation models: Claude, Amazon Nova, Titan, and others, through one API, with no servers to run and no model to host yourself.

Why managed matters here

- One client and one call gets you a production-grade model
- Models are billed per use, so there is nothing to provision
- Access is governed by your AWS account and IAM, not a separate vendor



THE CONVERSE API: ONE SHAPE FOR EVERY MODEL

Converse is the modern, model-agnostic way to call any Bedrock model. The same code works for Claude, Nova, and Titan, so swapping models is a one-line change.

The message format you will reuse all day

- A list of messages, each with a role and content
- Roles are user and assistant; system instructions ride alongside
- The exact same structure appears in RAG and tool use later

YOUR FIRST MODEL CALL

A Bedrock client, a model id, and a list of messages is all it takes to get a response from a live model.

```
import boto3
client = boto3.client("bedrock-runtime", region_name="us-east-1")

MODEL = "us.amazon.nova-lite-v1:0"

response = client.converse(
    modelId=MODEL,
    messages=[{"role": "user",
               "content": [{"text": "What is machine learning?"}]}],
)
print(response["output"]["message"]["content"][0]["text"])
```

Use case: the smallest possible call, the shape every later call extends.

MESSAGES ARE LISTS OF TYPED BLOCKS

That content field is a list, not a string, and each item is a typed block. It looks like extra ceremony for a text chat, but it is what lets one API carry text, images, and tool calls in the same shape.

Block types you will meet today

- text: the ordinary words of a message
- image: bytes for a vision-capable model
- toolUse and toolResult: the model's tool calls and your replies (Module 4)

Why a list:

Because content is a list of blocks, the same converse call handles a plain question, an image, or a tool result without changing shape.

SYSTEM PROMPTS: YOUR LAYER OF CONTROL

A system prompt is instructions given before the conversation starts. It sets the model's persona, tone, scope, and output format, and it is often more powerful than any amount of fine-tuning.

What a good system prompt does

- Turns a general model into a specialist for your task
- Is invisible to the end user; it is the developer's control layer
- Is the first place to look when a model behaves unexpectedly

Best practice:

Be explicit. 'Respond in exactly three sentences' beats 'be brief', and 'do not' is clearer than 'avoid'.

SHAPE BEHAVIOR WITH A SYSTEM PROMPT

The system prompt sets rules the model follows for the whole conversation. Here it forces a persona, a length limit, and an off-topic redirect.

```
system_prompt = """You are a concise data science tutor.  
- Answer in plain English, explain any jargon  
- Keep responses to three sentences maximum  
- If asked something off topic, redirect politely"""  
  
response = client.converse(  
    modelId=MODEL,  
    system=[{"text": system_prompt}],  
    messages=[{"role": "user", "content": [{"text": "What is overfitting?"}]],  
)
```

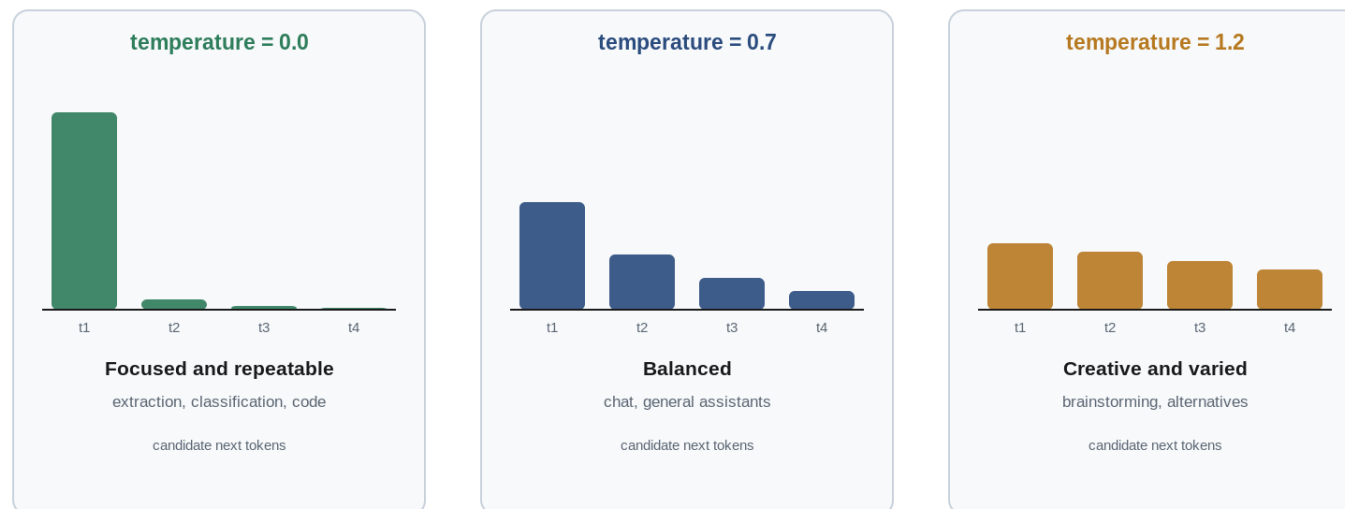
Use case: make one general model behave like a purpose-built assistant.

TEMPERATURE: FOCUSED OR CREATIVE

Temperature controls how the model chooses each next token. Low temperature makes it pick the likeliest option and stay consistent; high temperature spreads the odds and adds variety.

Temperature controls how the model picks the next token

Low temperature sharpens toward the likeliest token. High temperature flattens the odds.



Same prompt, different behavior. Start at 0 for anything that must be consistent.

Set it to 0 for extraction, classification, and code. Raise it only when you want variety.

CAPPING OUTPUT WITH MAX_TOKENS

Output tokens are the part you control least, because the model decides how much to write. `maxTokens` is the hard ceiling, and it doubles as a cost guard and a safety valve against runaway responses.

What to know

- `maxTokens` sets the ceiling on the response length
- If the model hits it, `stopReason` comes back as 'max_tokens'
- The text is cut off mid-thought, so set the cap with room to finish

Watch out:

Check `stopReason`. 'max_tokens' means the answer was truncated, not that the model was finished.

SET INFERENCE PARAMETERS

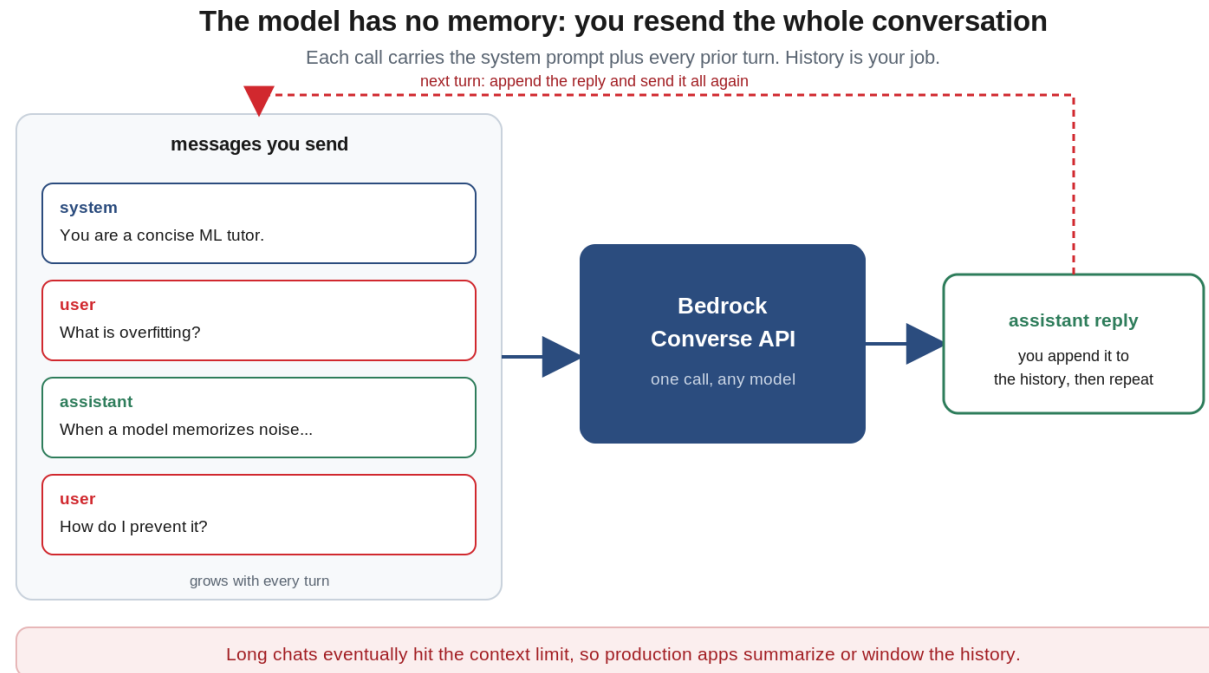
inferenceConfig is where you tune behavior: how random the output is, and how long it can run. These are the dials you reach for after the prompt itself.

```
response = client.converse(
    modelId=MODEL,
    messages=[{"role": "user", "content": [{"text": prompt}]}],
    inferenceConfig={
        "temperature": 0.2,    # low = focused and repeatable
        "topP": 0.9,
        "maxTokens": 300,      # hard cap on response length
    },
)
if response["stopReason"] == "max_tokens":
    print("Answer was truncated; raise maxTokens")
```

Use case: deterministic extraction at temperature 0, or a bounded, budget-safe reply.

THE MODEL HAS NO MEMORY

LLMs are stateless: they remember nothing between calls. To hold a conversation you resend the entire history every time, and maintaining that list is your job as the developer.



This is the core loop inside every chatbot you have ever used.

KEEP A CONVERSATION GOING

Because the model is stateless, you append each turn to a history list and send the whole thing on every call. That list is the conversation.

```
def chat(history, user_input):
    history.append({"role": "user", "content": [{"text": user_input}]})
    response = client.converse(modelId=MODEL, messages=history)
    reply = response["output"]["message"]["content"][0]["text"]
    history.append({"role": "assistant", "content": [{"text": reply}]})
    return reply, history

# the whole history rides along on every call
history = []
reply, history = chat(history, "I am a data analyst learning ML.")
reply, history = chat(history, "What should I study next?") # it remembers
```

Use case: any multi-turn assistant, from support bot to coding helper.

MANAGING HISTORY AT SCALE

Because you resend the whole conversation each turn, a long chat grows until it strains the window and the cost. Real assistants keep the history from growing without bound.

Two common strategies

- Sliding window: keep the system prompt plus the last N turns
- Summarize: replace old turns with a short running summary
- Either way, protect the system prompt and the most recent turns

Trade-off:

Trimming history changes what the model remembers. Test that the assistant still handles follow-up questions after a trim.

TWO HABITS: STREAMING AND COST AWARENESS

Two small habits separate a toy from a real integration: stream output so the app feels fast, and read token usage on every call so cost never surprises you.

Streaming

- `converse_stream` delivers tokens as they are generated
- The app feels responsive instead of frozen on long answers

Cost awareness

- Every response reports input and output token counts
- Output tokens cost more and the model decides how many
- A brevity rule in the system prompt directly cuts spend

STREAM THE RESPONSE TOKEN BY TOKEN

`converse_stream` returns events as the model generates them. You read the text deltas and show them immediately, which is what makes a chat feel alive instead of frozen.

```
stream = client.converse_stream(  
    modelId=MODEL,  
    messages=[{"role": "user",  
               "content": [{"text": "Explain transformers."}]}],  
)  
for event in stream["stream"]:  
    if "contentBlockDelta" in event:  
        delta = event["contentBlockDelta"]["delta"]  
        print(delta.get("text", ""), end="", flush=True)
```

Use case: a responsive chat UI that types the answer out live.

READ TOKEN USAGE AND COST

Every call returns the tokens it used. Reading them turns cost from a monthly surprise into a number you watch per request.

```
resp = client.converse(
    modelId=MODEL,
    messages=[{"role": "user", "content": [{"text": prompt}]}],
)
usage = resp["usage"]
cost = (usage["inputTokens"] / 1_000_000 * 0.06 +      # input rate
        usage["outputTokens"] / 1_000_000 * 0.24)     # output costs more
print(usage["inputTokens"], "in",
      usage["outputTokens"], "out", f"${cost:.6f}")
```

Use case: build a running cost meter or a per-request budget guard.

PICK THE RIGHT MODEL

Because Converse is model-agnostic, switching is a one-line change. Match the model to the task instead of paying for the biggest one everywhere.

Reach for a small model

- Classification and routing
- Short, factual answers
- High volume with tight latency
- Example: Nova Micro or Nova Lite

Reach for a bigger model

- Nuanced reasoning and judgment
- Long or ambiguous instructions
- Code and multi-step analysis
- Example: Nova Pro or a Claude model

POP QUIZ: BEDROCK

Your assistant won't stick to your three-sentence rule. Where do you look first, and why can't a system prompt enforce real security?



3 MINUTES

POP QUIZ: BEDROCK

Your assistant won't stick to your three-sentence rule. Where do you look first, and why can't a system prompt enforce real security?

Answer:

Start with the system prompt for the behavior, and make the rule explicit. But a system prompt cannot enforce security, because a determined user can talk the model into ignoring it.

- Behavior lives in the system prompt: 'exactly three sentences' beats 'be brief'
- Prompt injection can override instructions, so never trust it as access control
- Put hard limits in code and permissions, not in the prompt



MODULE 3: GROUNDING WITH RAG

RETRIEVAL AUGMENTED GENERATION



THE PROBLEM RAG SOLVES

Ask a raw model 'what is the default password for the QR-5000' and it will confidently invent one. It has never seen your manuals. RAG fixes this by handing the model the right document before it answers.

Why not just fine-tune or prompt harder

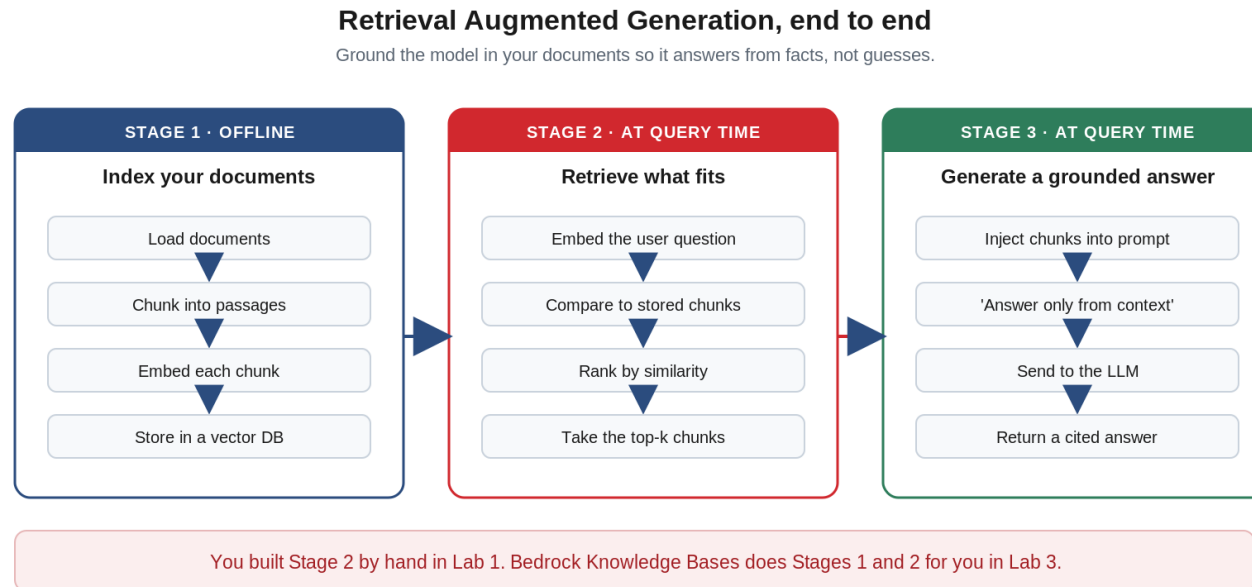
- Your documents change; retraining for every edit is impractical
- The model cannot cite a source it was never given
- Retrieval keeps answers current: update the document, not the model

In one line:

RAG = retrieve the relevant text first, then ask the model to answer using only that text.

RAG, END TO END

RAG has three stages: index your documents once, retrieve the relevant chunks at query time, then generate an answer grounded in those chunks.



You built the retrieve stage by hand in Lab 1; here AWS runs it for you.

FROM HAND-BUILT TO MANAGED: BEDROCK KNOWLEDGE BASES

In Lab 1 you embedded and searched documents yourself. In production you let Amazon Bedrock Knowledge Bases do the chunking, embedding, and indexing, so you write almost no infrastructure code.

What the managed service handles

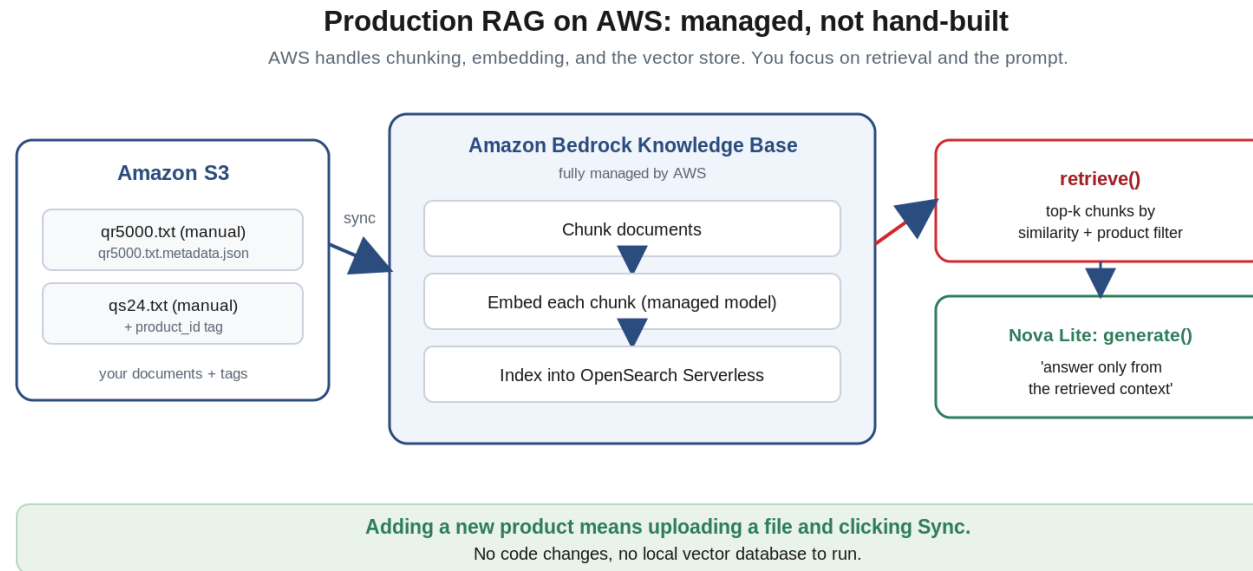
- Splitting documents into chunks and embedding each one
- Storing vectors in a managed OpenSearch Serverless index
- Keeping the index in sync with your S3 bucket

Why this matters:

You stop maintaining a vector database and focus on retrieval quality and the prompt, which is the real job.

PRODUCTION RAG ON AWS

Documents live in S3 with a small metadata file each. The Knowledge Base indexes them, you retrieve the top chunks, and a model generates the grounded answer.



Adding a new product is upload a file and click Sync, no code changes.

DOCUMENTS IN S3, SCOPED BY METADATA

Each manual gets a small metadata file tagging it with a product id. At query time you filter retrieval to one product, so a QR-5000 question never pulls a QS-24 chunk.

How the filter behaves

- The metadata tag works like a SQL WHERE clause on retrieval
- Without a filter you search across every product's manual
- With product_id set you get only that product's chunks

Watch out:

Metadata filtering is a relevance tool, not a security boundary. Do not use it to hide documents from authorized users.

CHUNKING: THE QUIET DECIDER OF RAG QUALITY

Before anything is embedded, each document is split into chunks, and that split decides what retrieval can even find. Chunk too big and results are vague; chunk too small and each piece loses the context that made it meaningful.

The balance to strike

- Large chunks retrieve fuzzy, off-target passages
- Tiny chunks lose the surrounding context that gives them meaning
- A little overlap between chunks keeps ideas from being cut in half

Why this matters:

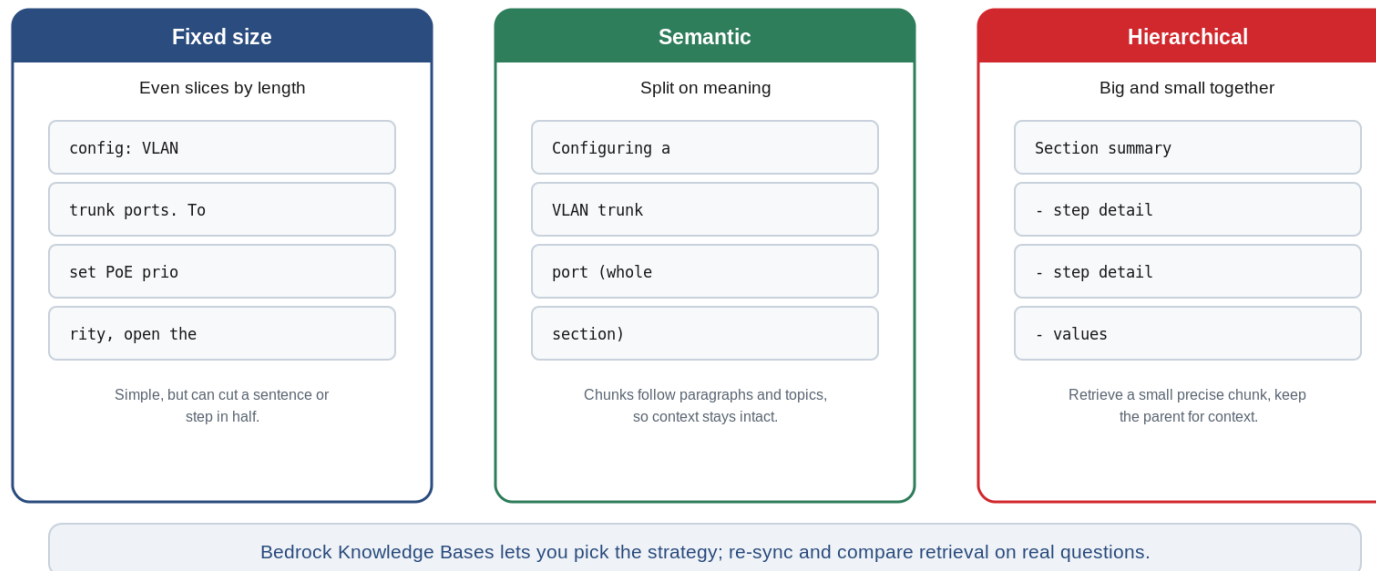
Teams often spend more time on chunking than on the prompt, because it moves answer quality the most.

THREE CHUNKING STRATEGIES

Bedrock Knowledge Bases offers a few strategies. Which one fits depends on how your documents are structured and how questions tend to span them.

Chunking quietly decides retrieval quality

How you split documents before embedding shapes what retrieval can find.



Change the strategy, re-sync, and compare retrieval on real questions before you commit.

RETRIEVE THE RIGHT CHUNKS

Retrieval is a pure vector search: your query becomes an embedding and is matched against the indexed chunks. No model runs yet, you are confirming the right text comes back.

```
def retrieve(query, product_id=None, top_k=4):
    config = {"numberOfResults": top_k}
    if product_id:
        # filter, like a SQL WHERE
        config["filter"] = {
            "equals": {"key": "product_id", "value": product_id}}
    response = BEDROCK_AGENT.retrieve(
        knowledgeBaseId=KB_ID,
        retrievalQuery={"text": query},
        retrievalConfiguration={"vectorSearchConfiguration": config},
    )
    return [{"text": r["content"]["text"], "score": r["score"]}
            for r in response["retrievalResults"]]
```

Use case: confirm retrieval quality before you spend a token on generation.

TUNING RETRIEVAL: TOP_K AND SCORE THRESHOLDS

Retrieval has dials too. How many chunks you pass, and how picky you are about their scores, together decide whether the model sees the right facts, too little, or too much noise.

The main knobs

- top_k: how many chunks to retrieve; more context, but more noise and cost
- Score threshold: drop chunks below a relevance floor
- Filters: scope by metadata so only the right product's chunks return

Key point:

More chunks is not always better. Passing weak, off-topic chunks can pull the answer off course.

POP QUIZ: CHUNKING

The manuals contain the details, yet answers are vague and retrieval scores are mediocre. What would you change first?



3 MINUTES

POP QUIZ: CHUNKING

The manuals contain the details, yet answers are vague and retrieval scores are mediocre. What would you change first?

Answer:

Start with chunking and retrieval settings, not the model. Mediocre scores mean the right text is not being retrieved cleanly.

- Try smaller or semantic chunks with a little overlap
- Adjust top_k and add a score threshold to cut noise
- Re-sync and re-test on questions with known answers



GROUNDING GENERATION: ANSWER ONLY FROM CONTEXT

Retrieval finds the text; generation writes the answer. The one instruction that makes RAG trustworthy is telling the model to answer using only the retrieved context, and to say so when the answer is not there.

What grounding buys you

- The model reads real documentation instead of guessing
- It can decline when the manuals do not cover the question
- You can show the source chunk behind every answer

Key point:

'Answer only from the context' is the line that turns a confident guesser into a reliable assistant. Test that it holds before shipping.

GENERATE A GROUNDED ANSWER

Stuff the retrieved chunks into the prompt as context, instruct the model to use only that context, and call the same Converse API from Module 2.

```
def generate(question, chunks):
    if not chunks:
        return "I do not have that in the product manuals."
    context = "\n\n---\n\n".join(c["text"] for c in chunks)
    prompt = f"""Answer using ONLY the context below.
If the answer is not there, say you do not have that information.

CONTEXT:
{context}

QUESTION: {question}"""
    resp = BEDROCK.converse(modelId=MODEL,
        messages=[{"role": "user", "content": [{"text": prompt}]}])
    return resp["output"]["message"]["content"][0]["text"]
```

Use case: a support answer that quotes the manual instead of improvising.

TWO WAYS TO ANSWER

Bedrock also offers a single call that retrieves and generates together. Choose based on how much control you need over the prompt.

retrieve() + generate()

- You write the generation prompt
- Full control of tone, format, filtering
- Post-process chunks before the model sees them
- Best when the prompt must be precise

retrieve_and_generate()

- One managed API call
- AWS formats citations for you
- Fastest path to a working answer
- Best for a quick, cited response

CITATIONS BUILD TRUST

A grounded answer is more trustworthy when it can show its work. Returning the source chunk behind each answer lets users verify it and lets you debug when it is wrong.

Why surface sources

- Users can check the claim against the real document
- You can see which chunk drove a wrong answer
- `retrieve_and_generate` returns citations for you automatically

The payoff:

Showing sources turns 'trust me' into 'here is where I found it', which is what makes a support tool credible.



EVALUATING A RAG SYSTEM

RAG shifts the failure mode from 'the model made it up' to 'retrieval brought back the wrong or stale text'. You cannot fix what you do not measure, so test before and after every change.

What to check

- Build a set of questions with known correct answers
- Measure whether the right chunk is retrieved, separately from the final answer
- Re-test after changing chunking, top_k, or the documents
- Watch for stale content after a source update

Best practice:

Test with questions where you know the answer before you put the system in front of anyone.

POP QUIZ: RAG

A product spec changed last week, but the assistant still quotes the old value. No code changed. What most likely went wrong?



3 MINUTES

POP QUIZ: RAG

A product spec changed last week, but the assistant still quotes the old value. No code changed. What most likely went wrong?

Answer:

The Knowledge Base was not re-synced after the document changed, so retrieval is serving the old chunk. Grounding is only as current as the last sync.

- Update the document in S3, then re-sync the Knowledge Base
- Grounding reduces hallucination but cannot fix stale sources
- Ownership is operational: someone must own the re-sync process



MODULE 4: AGENTS AND TOOL USE

WHEN THE MODEL NEEDS TO ACT



WHERE RAG STOPS

RAG is perfect for questions answered by static documents. It is the wrong tool for questions that need live data or that change the world, like current stock levels or creating a support ticket.

Two different jobs

- Documentation questions: how to configure a VLAN (RAG)
- Business questions: is it in stock, what does it cost (live data)
- Actions: create a ticket, update a record (writes)

TOOL USE: THE MODEL CALLS FUNCTIONS BY NAME

You describe a set of tools to the model. When it needs one, it does not run your code, it asks you to run it by name and waits for the result. You stay in control of every side effect.

The division of labor

- You define tools: a name, a description, and the inputs
- The model chooses which tool and what arguments to pass
- Your code runs the tool and hands back the result

The safety point:

The model never touches your database. It requests; your code decides whether and how to act.

DEFINE A TOOL

A tool definition is just a description the model reads: what the tool does and what inputs it needs. The model uses it to decide when and how to call.

```
TOOLS = [{
  "toolSpec": {
    "name": "check_stock",
    "description": "Check the current stock level for a product.",
    "inputSchema": {"json": {
      "type": "object",
      "properties": {
        "product_id": {"type": "string",
          "description": "Product ID, e.g. qr5000"}},
      "required": ["product_id"]}},
  }
}]
# get_price and create_ticket follow the same shape
```

Use case: expose exactly the actions you want, and nothing more.

IMPLEMENT THE TOOL EXECUTOR

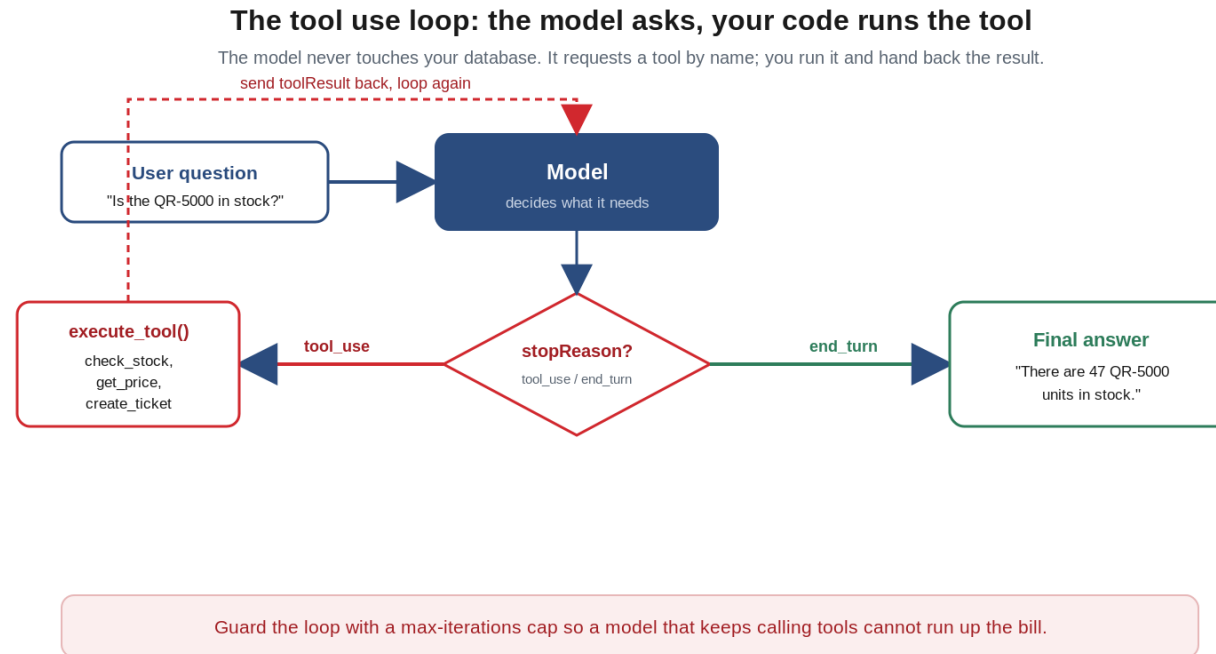
The model asks for a tool by name; this function is where the real work happens. It runs the query or write against DynamoDB and returns a short string the model can read.

```
def execute_tool(name, inputs):
    pid = inputs["product_id"]
    if name == "check_stock":
        item = products_table.get_item(Key={"product_id": pid})["Item"]
        return item["name"] + " has " + str(item["stock"]) + " in stock."
    if name == "get_price":
        item = products_table.get_item(Key={"product_id": pid})["Item"]
        return item["name"] + " costs $" + str(item["price"])
    if name == "create_ticket":
        tid = str(uuid.uuid4())[:8]
        tickets_table.put_item(Item={"ticket_id": tid,
                                     "product_id": pid, "issue": inputs["issue"],
                                     "status": "open"})
        return "Created ticket " + tid
```

Use case: read live inventory and write a real support ticket.

THE TOOL USE LOOP

The model may need several tools before it can answer. You loop: call the model, run any tool it asks for, hand back the result, and repeat until it signals it is done.



The loop exits when stopReason is end_turn.

RUN THE TOOL USE LOOP

Keep calling the model until it stops asking for tools. Each pass runs the requested tools and feeds the results back as the next message.

```
def run_tools(question):
    messages = [{"role": "user", "content": [{"text": question}]}]
    while True:
        response = bedrock.converse(modelId=MODEL, messages=messages,
                                     toolConfig={"tools": TOOLS})
        messages.append(response["output"]["message"])
        if response["stopReason"] == "end_turn":
            return response["output"]["message"]["content"][0]["text"]
        results = []
        for block in response["output"]["message"]["content"]:
            if "toolUse" not in block: continue
            t = block["toolUse"]
            out = execute_tool(t["name"], t["input"])
            results.append({"toolResult": {"toolUseId": t["toolUseId"],
                                           "content": [{"text": out}]}})
        messages.append({"role": "user", "content": results})
```

Use case: let a model gather what it needs before it answers.

THE MODEL PLANS, YOUR CODE GUARDS

Tool use hands the model real power: it can read your database and write to it. That makes your executor the safety gate. The model proposes; your code decides whether and how to act.

Guardrails that belong in code

- Validate every input before a read or write; never trust a model-supplied key
- Make writes safe to repeat, so a retried tool call cannot double-book
- Cap the loop with a maximum number of iterations
- Log every tool call so you can see what the model actually did

Watch out:

A tool use loop with no iteration cap can call tools forever and run up real spend. Add the cap before you ship.

ONE MODEL PER JOB: ROUTING

A single model doing everything is wasteful and hard to maintain. A better pattern is a cheap model that classifies each question, then routes it to the specialist pipeline that handles it best.

Why route

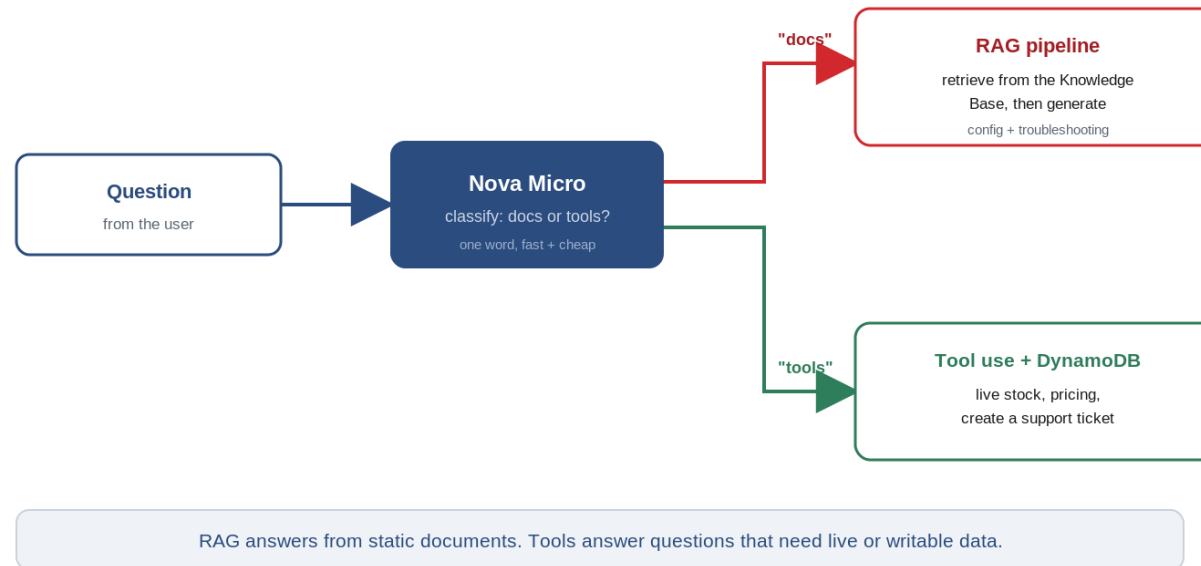
- A tiny model like Nova Micro is enough for a binary decision
- Each pipeline stays simple and testable on its own
- You spend the expensive model only where it earns its cost

ROUTE TO THE RIGHT SPECIALIST

Nova Micro reads the question and answers with a single word: docs or tools. That one word decides whether the RAG pipeline or the tool use pipeline handles it.

One cheap model routes each question to the right specialist

A tiny classifier is cheaper and easier to maintain than one giant model that does everything.



RAG answers from static documents; tools answer questions that need live or writable data.

CLASSIFY AND ROUTE

A tiny model decides the path, then the router hands off to RAG or to tool use. This is the whole assistant in a dozen lines.

```
def classify(question):
    prompt = f"""Reply with exactly one word.
docs  = product configuration or troubleshooting
tools = stock, pricing, or creating a ticket

Question: {question}"""
    resp = bedrock.converse(modelId=MODEL_MICRO,
        messages=[{"role": "user", "content": [{"text": prompt}]}])
    return resp["output"]["message"]["content"][0]["text"].strip().lower()

def answer(question, product_id=None):
    if classify(question) == "tools":
        return run_tools(question)          # live data
    chunks = retrieve(question, product_id) # docs -> RAG
    return generate(question, chunks)
```

Use case: one entry point that sends each question to the right engine.

STRUCTURED OUTPUT FOR RELIABLE ROUTING

A one-word answer works, but it gives you no signal when the model is unsure. Asking for structured output, like JSON with a confidence score, turns an opaque guess into something your code can reason about.

From one word to a decision you can trust

- Ask for JSON: a category plus a confidence from 0 to 1
- Route to tools only above a confidence threshold
- Default to RAG, the safer path, when confidence is low

Also helps:

Few-shot examples in the classifier prompt also sharpen borderline cases; show it three examples per category.

FROM TOOL USE TO AGENTS

Put the pieces together and you have the core of an agent: a model that plans, calls tools in a loop, reads the results, and keeps going until a goal is met. Tool use is the mechanism; an agent is what you build with it.

What makes it agentic

- It chooses actions, it is not following a fixed script
- It loops: act, observe the result, decide the next step
- It works toward a goal across several steps

The trade-off:

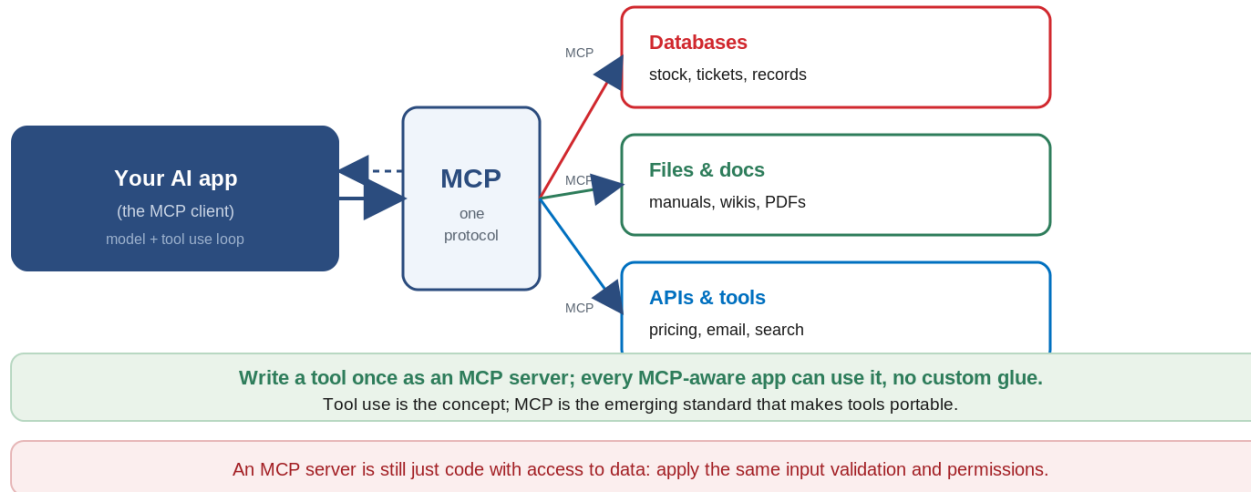
More autonomy means more that can go wrong unattended. Keep the guardrails: input checks, iteration caps, and logging.

MCP: A STANDARD WAY TO EXPOSE TOOLS

In Lab 4 you hand-wired each tool into your app. That works, but every app re-implements the same glue. The Model Context Protocol is an open standard that lets a tool be written once and reused by any compatible client.

MCP: one standard way to connect models to tools and data

You hand-wired each tool in Lab 4. MCP standardizes that wiring so any client can reuse any server.



Tool use is the concept you learned; MCP is the emerging standard that makes tools portable.

POP QUIZ: AGENTS

Some questions, like 'What is the QR-5000?', could be answered by RAG or by tools. How do you keep the router from guessing wrong on ambiguous questions?



3 MINUTES

POP QUIZ: AGENTS

Some questions, like 'What is the QR-5000?', could be answered by RAG or by tools. How do you keep the router from guessing wrong on ambiguous questions?

Answer:

Give the classifier more to go on, and design a safe default for when it is unsure rather than forcing a hard choice.

- Add a few example questions per category to the classifier prompt
- Ask for a confidence score and default to RAG when it is low
- Add a 'both' path that runs each pipeline and combines the answers



PUTTING IT TOGETHER

FROM PROTOTYPE TO PRODUCT

SERVER STATUS

CPU

22 %

Memory

57 %

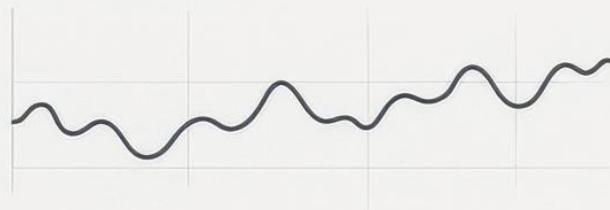
Disk

72 %

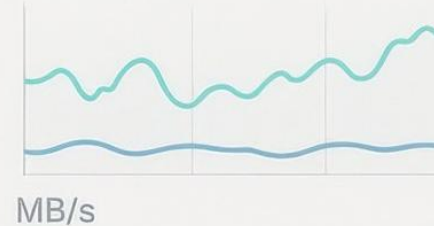
Network

134 MB/s

CPU Usage



Traffic



Memory Usage



THE FINISHED ASSISTANT

Everything today composes into one support assistant: a cheap classifier routes each question, RAG answers from the manuals, and tool use handles live inventory and tickets. That is a shippable product, built from parts you now understand.

What each piece contributed

- Tokens and embeddings: how retrieval finds the right text
- Bedrock Converse: the one call behind every model interaction
- RAG: grounded answers from your own documents
- Tool use and routing: live data and actions, safely

WHERE THIS SHOWS UP AT WORK

The same pattern, retrieve then generate, optionally with tools, powers a huge range of internal products. Once you can build the QuantumNet assistant, you can build these.

Common applications

- Customer and IT support assistants grounded in real docs
- Internal knowledge search over policies, runbooks, and wikis
- Document question answering for contracts, reports, and manuals
- Data-backed copilots that read dashboards and open tickets



RESPONSIBLE AI FOR LLM APPS

These systems are powerful and fallible. The practices below run through every lab today and are what separate a demo from something you can put in front of users.

Carry these into every build

- Grounding reduces hallucination but never eliminates it; test known answers
- System prompts and metadata filters are not security boundaries
- Validate every model-supplied input before a write or an action
- Keep sources fresh; a stale index gives confident wrong answers
- Do not send real personal data or secrets into prompts; calls are logged

The rule to remember:

Put hard limits in code and permissions. Use the model for language and judgment, not for enforcing rules.

KEY TAKEAWAYS

You went from how a model reads a single sentence to a routed assistant that reads documents and touches live data. The parts are simple; the skill is composing them responsibly.

- LLMs read tokens and compare meaning as vectors; that powers all retrieval
- Bedrock's Converse API is one stateless call that works across models
- RAG grounds answers in your documents: retrieve first, then generate
- Tool use lets a model act; a cheap classifier routes each question
- Ground it, validate it, keep it fresh, and enforce limits in code

Well Done

You can now build real AI applications with Bedrock.



Congratulations

"Develop a passion for learning."